

Group 5 Project Proposal

TabDDPM: Using Diffusion Models to Generate Synthetic Patient Data for Hospital Readmission Prediction

Taneir Arani
Jiten Bhalavat
Rohith Mandla
Anum Sagheer
Simi Shrivastava

1. Introduction

Patient readmission means whether a patient would come back in less than a span of 30 days. Readmission within such a short span is expensive and also indicates that something in the patient's treatment went wrong. Predicting it early can save lives and reduce costs.

While machine learning can help solve this, there is a problem with data. In this dataset, 11% of patients were readmitted within 30 days and 89% were fine. When you train a classifier on this data, it learns to say "not readmitted" for almost everyone because that is correct 89% of the time. It never really learns what a readmission patient looks like. So it misses the exact patients that matter most. This is called class imbalance. It is one of the most common and most dangerous problems in medical machine learning.

One way of fixing this, is to generate synthetic examples of the rare class, that are fake but realistic patients who were readmitted, and add them to the training data so the model sees enough examples to learn from. Our project aims on learning whether a diffusion model, specifically TabDDPM (Tabular Denoising Diffusion Probabilistic Model), can generate better synthetic patients than existing methods result in better accuracy.

This problem matters beyond the classroom. Companies like Syntegra, Gretel.ai, and MDClone are built entirely around synthetic patient data generation and are deployed in real hospitals today. The NHS in the UK has funded synthetic data projects. This is an active, funded, real-world problem.

2. Background and Related Work

Before our project, researchers have tried two main approaches to fix class imbalance with synthetic data.

SMOTE (Synthetic Minority Over-sampling Technique, Chawla et al. 2002) is the simplest method. It works by taking a minority class patient, finds their nearest neighbors in the dataset, and creates a new fake patient by averaging between them. It requires no training and is fast. But it just interpolates between existing patients and it cannot create anything outside what it has already seen, and it has no understanding of the underlying data distribution.

CTGAN (Conditional Tabular GAN, Xu et al. NeurIPS 2019) uses a Generative Adversarial Network to generate synthetic tabular rows. It is more powerful but GANs are known to be unstable to train. The generator and discriminator compete against each other and one can dominate the other, causing training to produce useless outputs.

DDPM (Denoising Diffusion Probabilistic Models, Ho et al. NeurIPS 2020) introduced a new kind of generative model that works by learning to reverse a gradual noise-adding process. Instead of two competing networks, one network learns one simple job, predict how much noise was added to the data. This is dramatically more stable than a GAN.

TabDDPM (Kotelnikov et al. ICML 2023) takes the DDPM framework and adapts it specifically for tabular data, the kind of structured row-and-column data found in hospital records. It handles both continuous columns like age and glucose, and categorical columns like gender and diagnosis codes. It also supports class conditioning, and it is easier to tell which type of patient to generate. This is the method our project uses.

3. Problem Statement

Can a diffusion model learn what high-risk diabetic patients look like well enough to generate fake but realistic ones and will adding those fake patients to the training data help a classifier get better at catching the real ones?

Our dataset contains information from 101,766 real diabetic patients from 130 hospitals in the United States. From all these patients, only 11% are readmitted within 30 days. This is exactly the group we are concerned about, but because they are so few in number, a standard classifier does not learn to classify them very well.

We train a TabDDPM model on this dataset to generate new synthetic patients that resemble these high-risk readmittance patients. We then add these synthetic patients to the dataset check if this classifier get better at classifying real high-risk readmittance patients. We compare this to three other approaches: no fix at all, SMOTE, and CTGAN.

4. Dataset

Our dataset is the [Diabetes 130-US Hospitals](#) dataset, free to download from Kaggle.

It contains records from 101,766 real patients treated at 130 US hospitals between 1999 and 2008. Each patient has 50 pieces of information recorded, things like their age, gender, race, how long they stayed in the hospital, how many medications they were on, their insulin status, their blood sugar test result, and their primary diagnosis. The last column is what we are trying to predict, whether the patient returned to the hospital within 30 days or not.

Some of these 50 columns are numerical, like the age of the patients and the length of stay. Some are categorical, like gender and type of diagnosis. Our model needs to deal with both, and that is one of the things that makes dealing with medical data tricky.

Out of all 101,766 patients, only 11% were readmitted within 30 days, and the rest, 89%, were not. This is what we are trying to solve.

We need to clean the data before we train anything on it, converting categorical data to numerical data that the model can understand, scaling all numerical data, and dividing the data into 80% training and 20% testing, making sure that both sets have the same ratio.

5. Proposed Approach

Our project has three parts, building the model, generating fake patients, and testing whether those fake patients actually help.

Part 1: Training TabDDPM

We build TabDDPM in PyTorch based on the paper's design. Training happens in two phases.

In the first phase we take a real patient row and slowly add random noise to the numbers, step by step, 1000 steps total. By the end the original values are completely gone, it just looks like random garbage. This phase involves no learning at all. We are just doing math.

In the second phase, we train another neural network that tries to reverse what we did in the previous phase. It tries to look at the noisy patient data, look at what step it is on, and try to guess what kind of noise was added. We then look at the actual noise that was added and the difference between that and the guess that the network makes, and that is the loss.

The network becomes better and better at predicting the noise, and eventually, it becomes so good that it can reverse the whole process of corruption. The loss that we use here is straightforward and stable, and it behaves like any standard regression loss.

For the columns that are categories, like gender and diagnosis type, we cannot add Gaussian noise because they are not numerical data. So, we use another type of loss called multinomial diffusion.

We also let the model know what class each patient is in, so that it learns what readmission patients look like and what non-readmission patients look like.

Part 2: Generating Synthetic Patients

Once the model is trained, generating a new patient is simple. We start with a completely random row of numbers, no real patient involved at all, tell the model to generate a readmission patient, and run the reverse process 1000 steps. Each step the numbers become slightly more patient-like. After 1000 steps we have a brand new synthetic patient who looks like a real high-risk readmission case but never existed in our dataset.

We generate enough of these to run experiments at different augmentation levels, adding 50%, 100%, and 200% more minority class patients, to see how much synthetic data is needed.

Part 3: Baselines and Evaluation

To know if TabDDPM is actually useful we compare it against three alternatives. First, doing nothing, train on the original imbalanced data and see how bad it is. Second, SMOTE, a simple method that creates fake patients by averaging between real ones. Third, CTGAN, a GAN-based synthetic data generator, also available as a ready-made library.

For each method we retrain an XGBoost classifier on the augmented dataset and test it on the real held-out patients. Our main measurement is F1 score on the readmission class, how well does the classifier find the rare high-risk patients. We also report AUC-ROC. We repeat every experiment 5 times with different random seeds and report the average and standard deviation so our results are reliable.

6. Evaluation Plan

Our main measurement is F1 score on the readmitted patients specifically, not overall accuracy. F1 tells us how well the classifier is finding the rare high-risk cases after we add synthetic data. This is the number we care about most.

Our secondary measurement is AUC-ROC, which tells us how well the classifier ranks high-risk patients above low-risk ones overall.

Before we even run the classifier, we want to verify that the patients we generated look realistic. We want to verify that the distribution of the real patients and the distribution of the patients we generated look similar. We do that in three ways. The first way is that we plot each column side by side, real patients vs. generated patients, and verify that they look similar in distribution. The second way is that we calculate Jensen-Shannon divergence, which mathematically measures how similar two distributions are. The closer to zero, the more similar they are. The third way is that we calculate t-SNE, where we reduce all 50 columns down to 2 dimensions and plot that to verify that the real and generated patients look similar.

We also run one harder test, we train the classifier entirely on synthetic patients and test it on real ones. If it still works reasonably well, that means our generated data genuinely captured the patterns of real patients. This is the most honest test we can run.

Finally we experiment with how much synthetic data to add, 50%, 100%, and 200% of the minority class size. This tells us whether more fake patients always helps or whether there is a point where adding too many starts to hurt.

7. Scope and Limitations

Here is what we will do. We will implement TabDDPM in PyTorch, train it on the diabetes dataset, generate synthetic patients, and compare the results against SMOTE and CTGAN. We will report a full evaluation with all the metrics described above.

Here is what we will not do. We are not building a hospital tool. We are not connecting to any real medical system. We are not adding any privacy protection mechanisms. We are not testing on multiple datasets.

This is a research project. The goal is to find out whether a diffusion model generates better synthetic patients than existing methods, and whether that leads to a better classifier. That is the question we are answering,

8. Team and Timeline

Anum: Forward Process and Noise Scheduling

Responsible for the very first step of how the diffusion model works. She writes the noise scheduler, the mathematical schedule that controls how much Gaussian noise gets added to a patient row at each of the 1000 steps, following a specific formula from the paper that slowly increases noise until the original patient values are completely unrecognizable. She also writes the forward process function, the code that takes a real patient row and a step number and returns the correctly noised version of that row.

She handles how noise works differently for categorical columns like gender and diagnosis codes. These cannot have Gaussian noise added to them because they are not continuous numbers so Anum implements multinomial diffusion for these columns, which randomly replaces category labels instead of adding continuous noise.

Jiten Bhalavat: Neural Network Architecture

This is the neural network that looks at a noisy patient row and tries to figure out what noise was added to it. He will implement this as an MLP (multi-layer perceptron), a stack of fully connected layers with activation functions between them. He decides how many layers the network has, how wide each layer is, and how information flows through it from input to output.

This also includes timestep embedding, the network needs to know which of the 1000 noise steps it is looking at, because the amount of noise at step 50 is very different from step 900. He writes the code that converts the step number into a vector the network can understand and use. The final output is a clean working model class in PyTorch that takes a noisy patient row and step number as input and outputs a noise prediction, ready to be handed to Rohith for training.

Simi Shrivastava: Class Conditioning and Generation

She teaches the model the difference between a readmitted patient and a non-readmitted patient. She writes the conditioning code, the part that takes the readmission label, converts it into a vector, and injects it into the network so the model learns two separate distributions during training. Without this, the model generates random patients with no control over which class they belong to.

She also writes the generation script, the code that starts from a completely random row of numbers, sets the class condition to readmitted, and runs the trained model backward 1000 steps until a realistic synthetic patient appears. This is the most visible output of the whole project.

Rohith Mandla: Data Pipeline and Training Loop

He has two jobs and is the first person to start working since the whole team depends on their data being ready.

His first job is data preparation. He takes the raw dataset, cleans it up, converts all text columns into numbers the model can read, scale all numeric values to a standard range, and split everything into training and test sets. This clean prepared data is what Anum needs to build and test the noise scheduling code on real patient rows, so he delivers this first.

His second job is the training loop. Once Jiten has finished building the neural network, Rohith takes that network and writes the training process around it, the loss function that measures how wrong the model's noise prediction is, the optimizer that adjusts the weights to make it less wrong, and the code that runs the whole thing through the data repeatedly epoch by epoch until the model actually learns.

Taneir Arani: Baselines, Convergence, and Evaluation

She monitors the training to make sure the model is actually learning, loss going down, no crashes, results consistent across runs. She also runs experiments to find the best settings for the model like learning rate, batch size, and number of layers.

On top of that she implements the three comparison methods, the no-fix baseline, SMOTE, and CTGAN, and runs all the experiments that answer the main question: does TabDDPM actually perform better? She records all the F1 and AUC numbers across all conditions and all 5 random seeds.

She also runs the test where the classifier is trained entirely on synthetic patients and tested on real ones. Finally she runs experiments at three different augmentation levels, adding 50%, 100%, and 200% more minority class patients, to find out how much synthetic data actually helps and whether adding more always leads to better results or not.

All Five Members: Research, Report, Presentation, and Visualizations

Everything written, presented, and visualized is shared equally across the team. Every member reads all the papers and understands the whole project. Each member writes the report section for their own component and creates the visuals that go with it, Anum visualizes how noise builds up over steps, Jiten draws the network architecture, Simi shows examples of real vs. generated patients, Rohith plots the training loss curves, and Taneir produces the final results charts and comparison plots. And if anyone gets stuck at any point, the rest of the team helps.

9. References

- [1] Kotelnikov, A., Baranchuk, D., Rubachev, I., & Babenko, A. (2023). TabDDPM: Modelling Tabular Data with Diffusion Models. ICML 2023. <https://arxiv.org/abs/2209.15421> (*main paper*)
- [2] Ho, J., Jain, A., & Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. NeurIPS 2020. <https://arxiv.org/abs/2006.11239>

- [3] Xu, L., Skoularidou, M., Cuesta-Infante, A., & Veeramachaneni, K. (2019). Modeling Tabular Data using Conditional GAN. NeurIPS 2019. <https://arxiv.org/abs/1907.00503>
- [4] Chawla, N., Bowyer, K., Hall, L., & Kegelmeyer, W. (2002). SMOTE: Synthetic Minority Over-sampling Technique. JAIR 2002. <https://arxiv.org/abs/1106.1813>